Bubblesort & Running-Time Practice Problems

CS-UY 2413 — Design and Analysis of Algorithms

Covers Lectures 1–2

```
1 BUBBLESORT(A, n)
2 1  for i = 1 to n - 1
3 2       for j = n downto i + 1
4 3           if A[j] < A[j - 1]
5 4               exchange A[j] with A[j - 1]
```

Conventions (as in lecture): arrays are indexed from 1; $A[i : j]$ means all entries from $A[i]$ through $A[j]$ inclusive. For a loop `for x = a to b`, the loop-control line executes $b - a + 2$ times (that's $b - a + 1$ "true" tests plus one final "false" test), while the loop body executes $b - a + 1$ times. **Loop invariant** (Lecture 1): at the start of each outer-loop iteration, $A[1 : i-1]$ contains the $i-1$ smallest elements of the original A, in sorted order, and $A[i : n]$ contains the rest.

Problem 1 : Trace it, and read the invariant

(a) Let $A = [9, 2, 6, 4, 1]$ (so $n = 5$). Show the contents of A at the **start of each outer-loop iteration** ($i = 1, 2, 3, 4$), and the final sorted array.

(b) Now suppose $n = 8$ and Bubblesort is about to begin the iteration for $i = 5$. According to the loop invariant, what do you know about $A[1 : 4]$ and $A[5 : 8]$? Be precise in the answer you give. Does the invariant give you exact *values*, or something weaker?

Solution. (a) For each fixed i, the inner loop bubbles the minimum of $A[i : n]$ down to position i.

$i = 1$: $[9, 2, 6, 4, 1] \to [9, 2, 6, 1, 4] \to [9, 2, 1, 6, 4] \to [9, 1, 2, 6, 4] \to [1, 9, 2, 6, 4]$

$i = 2$ (start $[1, 9, 2, 6, 4]$): $j = 5$: $4 < 6$, swap $\to [1, 9, 2, 4, 6]$; $j = 4$: $4 < 2$? no; $j = 3$: $2 < 9$, swap $\to [1, 2, 9, 4, 6]$. End: $[1, 2, 9, 4, 6]$.

$i = 3$ (start $[1, 2, 9, 4, 6]$): $j = 5$: $6 < 4$? no; $j = 4$: $4 < 9$, swap $\to [1, 2, 4, 9, 6]$. End: $[1, 2, 4, 9, 6]$.

$i = 4$ (start $[1, 2, 4, 9, 6]$): $j = 5$: $6 < 9$, swap $\to [1, 2, 4, 6, 9]$. Loop ends.

i	A at *start* of iteration
1	$[9, 2, 6, 4, 1]$
2	$[1, 9, 2, 6, 4]$
3	$[1, 2, 9, 4, 6]$
4	$[1, 2, 4, 9, 6]$
final	$[1, 2, 4, 6, 9]$

(b) $A[1:4]$ contains the 4 smallest elements of the **original** array, in sorted order. $A[5:8]$ contains the remaining 4 elements, but the invariant makes **no claim about their order**. So it tells you *which* elements are where and that the left part is sorted. It does **not** give the exact values (unless you know the original array), and nothing about the order of the right part.

Problem 2 : Counting comparisons and swaps

(a) As an exact function of n (not big-O), how many times does line 3 (`if A[j] < A[j-1]`) execute, regardless of input?

(b) Give an input for which `BUBBLESORT` performs **zero** swaps.

(c) Give an input for which it performs the **maximum possible** number of swaps, and state that number.

Solution. (a) For each i from 1 to $n-1$, the inner loop runs $n-i$ values of j, one comparison each:

$$\sum_{i=1}^{n-1}(n-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

This does not depend on the input at all (note that the number of swaps does).

(b) Any already-sorted array, e.g. $A = [1, 2, \ldots, n]$: every comparison is false, so line 4 never runs.

(c) A strictly decreasing array, e.g. $A = [n, n-1, \ldots, 1]$: every comparison triggers a swap, so the number of swaps equals the number of comparisons from (a): $\frac{n(n-1)}{2}$. This is also the max possible, since swaps can never exceed comparisons.

Problem 3 : Correctness subtleties: duplicates and stability

(a) The lecture assumes all elements of A are distinct. If instead $A = [3, 1, 3, 2]$ has duplicates, does `BUBBLESORT` still sort it correctly?

(b) A sort is **stable** if elements that compare equal keep their original relative order. This means that if one equal element appeared *before* another in the input, it must still appear before it in the output. For example, suppose the input has two elements with the same value, say two 4's, and we temporarily label them 4_a (the one that appears first, further left) and 4_b (the one that appears second) just so we can tell them apart. A stable sort is required to output them in the order $\ldots, 4_a, \ldots, 4_b, \ldots$. In other words, it is never allowed to output 4_b before 4_a, even though from the sort's perspective the two values are tied and either order would technically still count as sorted. Is this version of `BUBBLESORT` stable? Justify using line 3 specifically.

Solution. (a) Yes, it still sorts correctly ($[3, 1, 3, 2] \to [1, 2, 3, 3]$).

i	A at *start* of iteration
1	$[3, 1, 2, 3]$
2	$[1, 3, 2, 3]$
3	$[1, 2, 3, 3]$
final	$[1, 2, 3, 3]$

(b) Yes, stable. Line 3 swaps only when $A[j] < A[j-1]$ (strict), so two adjacent equal elements are never swapped by any single comparison. Since equal elements can only have their relative order changed by being swapped *directly past each other* (not past a third, different element), and that never happens, their original relative order is preserved throughout. (Using $\leq$ instead would still sort correctly but could break stability.)

Problem 4 : Modifying the algorithm

(a) Modify the `BUBBLESORT` algorithm so that it stops as soon as a full inner-loop pass makes no swaps. Give the modified pseudocode, and state the best-case running time (as a function of n) and on what kind of input it's achieved. Compare to the unoptimized best case from Problem 2(a).

(b) Change a *single line* of the original pseudocode so `BUBBLESORT` sorts into non-increasing order instead. Explain why your change works, in terms of the loop invariant.

Solution. (a)

```
1  BUBBLESORT-OPT(A, n)
2  1  for i = 1 to n - 1
3  2       swapped = FALSE
4  3       for j = n downto i + 1
5  4           if A[j] < A[j - 1]
6  5               exchange A[j] with A[j - 1]
7  6               swapped = TRUE
8  7       if swapped == FALSE
9  8           return
```

Best case: already-sorted input. The very first pass ($i = 1$) does $n - 1$ comparisons, finds no swaps, and returns which is $O(n)$, linear. This beats the unoptimized version, which *always* does $n(n-1)/2 = \Theta(n^2)$ comparisons no matter the input (Problem 2(a)), since it has no way to detect that no work remains.

(b) Change line 3 to `if A[j] > A[j-1]`. The original invariant relies on bubbling the *smallest* remaining element into position i each outer iteration. Flipping the comparison flips which element "wins" each swap, so now the *largest* remaining element gets bubbled into position i instead. The invariant becomes: $A[1 : i-1]$ holds the $i-1$ largest elements, in non-increasing order. At termination the whole array is sorted largest to smallest.

Problem 5 : Debugging via the invariant

A student proposes reversing the inner loop's direction:

```
1  BUBBLESORT-BAD(A, n)
```

```
1  for i = 1 to n - 1
2      for j = i + 1 to n
3          if A[j] < A[j - 1]
4              exchange A[j] with A[j - 1]
```

(a) Find an input on which this does *not* sort correctly, and trace it to show the bug.

(b) Explain in terms of the loop invariant why running j upward (instead of downward) breaks correctness. What does the original inner loop accomplish that this version fails to?

Solution. (a) Try $A = [2, 3, 1]$, $n = 3$. $i = 1$: $j = 2$: $A[2]$=3 vs $A[1]$=2, not less, no swap; $j = 3$: $A[3]$=1 vs $A[2]$=3, $1 < 3$, swap $\rightarrow [2, 1, 3]$. $i = 2$: $j = 3$: $A[3]$=3 vs $A[2]$=1, not less, no swap. Final: $[2, 1, 3]$ which is **not sorted**.

(b) In the **original** (downward) version, j counts down from n to $i + 1$. As shown in the Lecture 1 correctness proof, whichever value is the true minimum of $A[i : n]$ gets "picked up" the moment j reaches it (compared against something larger immediately to its left) and then keeps winning every subsequent comparison as j keeps decreasing — sliding one position further left on *each* remaining step of the pass — until j reaches $i + 1$ and the minimum lands in position i. So position i's final value is the result of a whole chain of comparisons, against every element of $A[i : n]$ that lay between the minimum's original position and position i.

In the **modified** (upward) version, j counts up from $i + 1$ to n. Position i is only ever involved in the very *first* comparison of the pass (when $j = i + 1$, comparing $A[i + 1]$ against $A[i]$) and is never touched again. Once j advances to $i + 2, i + 3, \ldots$, index i is behind the scan and is simply left alone. So whatever ends up at position i is just $\min(A[i], A[i + 1])$, using only the *two* elements that started there. It has no mechanism for "finding out" that some smaller value might be sitting further right, at position $i + 2$ or beyond, because that comparison never happens.

This is exactly the bug in the trace above: with $A = [2, 3, 1]$, $i = 1$, the pass compares $A[2] = 3$ against $A[1] = 2$, finds $3 \not< 2$, and then moves on to $j = 3$ which leaves position 1 alone for the rest of the pass. But the true minimum of $A[1 : 3]$ is 1, sitting at position 3, and position 1 is never compared against it. The invariant "$A[1 : i - 1]$ holds the $i - 1$ smallest elements, sorted" therefore fails to be established at $i = 2$, which is why the final array comes out unsorted.

Part II: Lecture 2 : Exact Running Time & Asymptotic Notation

Problem 6 : Exact operation counting

(a) Exactly how many asterisks are printed, as a function of n? Then give the Θ notation.

```
for i = 1 to n
    for j = 1 to n - i
        print "*"
```

Solution. (a) For fixed i, the body runs $n-i$ times. Total: $\sum_{i=1}^{n}(n-i) = \sum_{k=0}^{n-1} k = \frac{n(n-1)}{2} = \Theta(n^2)$.

Problem 7 : Asymptotic notation (homework-style)

(a) Recall $O(g(n)) = \{f(n) : \exists c > 0, \exists n_0 > 0 \text{ s.t. } \forall n \geq n_0,\ 0 \leq f(n) \leq c\, g(n)\}$. Let $f(n) = 5n + 3$, $g(n) = n^2$. Prove $f(n) = O(g(n))$ by exhibiting specific c, n_0 and verifying.

(b) True or false (no explanation required, but check yourself): (i) $5n^2 + 2n = O(n^2)$ (ii) $n^3 = O(n^2)$ (iii) $2^n = O(n^{10})$ (iv) $n \log n = o(n^2)$ (v) $n! = \Omega(2^n)$

Solution. (a) Take $c = 8$, $n_0 = 1$. For $n \geq 1$: $n \leq n^2$ and $1 \leq n^2$, so $5n + 3 \leq 5n^2 + 3n^2 = 8n^2$. Hence $5n + 3 \leq c\, g(n)$ for all $n \geq n_0 = 1$.

(b) (i) **True** (drop lower-order term and constant: $5n^2 + 2n \leq 6n^2$ for $n \geq 1$). (ii) **False** ($n^3/n^2 = n \to \infty$, no constant c works). (iii) **False** (exponentials eventually dominate every polynomial). (iv) **True** ($\frac{n \log n}{n^2} = \frac{\log n}{n} \to 0$). (v) **True** ($n! \geq (n/2)^{n/2}$ eventually exceeds any fixed exponential base).